testBench - XOR Arbiter PUF (Verilog)

```
`timescale 1ns/1ps

module xor_apuf_tb;

reg clk;
reg reset;
wire final_response;

// Internal signal access wires
wire [63:0] challenge;
wire [63:0] perm;
wire [63:0] stage1, stage2, stage3;
wire [14:0] sr1, sr2, sr3;

// DUT
xor_apuf uut (
    .clk(clk),
    .reset(reset),
    .final_response(final_response)
);

// ================================
// CONNECT INTERNAL SIGNALS
// ================================
assign challenge = uut.challenge;
assign perm      = uut.perm;
assign stage1    = uut.stage1;
assign stage2    = uut.stage2;
assign stage3    = uut.stage3;
assign sr1       = uut.sr1;
assign sr2       = uut.sr2;
assign sr3       = uut.sr3;

// ================================
// CLOCK
// ================================
always #5 clk = ~clk;

// ================================
// STIMULUS
// ================================
initial
begin
    clk = 0;
    reset = 1;

    #20 reset = 0;

    #2000;

    $stop;   // For ISE
end

endmodule




module xor_apuf(
input clk,
input reset,
output final_response
);

// =====================
// 1. Challenge Generator
// =====================
reg [63:0] challenge;

always @(posedge clk or posedge reset)
begin
```

```
    if(reset)
        challenge <= 64'hACE11234ABCD5678;
    else
        challenge <= {challenge[62:0], challenge[63]^challenge[62]};
end

// =====================
// 2. Permutation Block
// =====================
wire [63:0] perm;

assign perm = {
challenge[3],challenge[7],challenge[12],challenge[20],
challenge[1],challenge[9],challenge[15],challenge[22],
challenge[0],challenge[18],challenge[6],challenge[25],
challenge[30],challenge[10],challenge[11],challenge[19],
challenge[4],challenge[8],challenge[16],challenge[28],
challenge[2],challenge[5],challenge[13],challenge[21],
challenge[23],challenge[24],challenge[26],challenge[27],
challenge[29],challenge[31],challenge[32],challenge[33],
challenge[34],challenge[35],challenge[36],challenge[37],
challenge[38],challenge[39],challenge[40],challenge[41],
challenge[42],challenge[43],challenge[44],challenge[45],
challenge[46],challenge[47],challenge[48],challenge[49],
challenge[50],challenge[51],challenge[52],challenge[53],
challenge[54],challenge[55],challenge[56],challenge[57],
challenge[58],challenge[59],challenge[60],challenge[61],
challenge[62],challenge[63],challenge[14],challenge[17]
};

// =====================
// 3. Delay Chains (PDL)
// =====================
wire [63:0] stage1, stage2, stage3;

genvar i;
generate
for(i=0;i<64;i=i+1)
begin: delay_chain

assign stage1[i] = perm[i] ? ~challenge[i] : challenge[i];
assign stage2[i] = perm[i] ? challenge[i] : ~challenge[i];
assign stage3[i] = perm[i] ? (challenge[i]^perm[i]) : challenge[i];

end
endgenerate

// =====================
// 4. Arbiter (Decision)
// =====================
wire r1, r2, r3;

assign r1 = stage1[63] ^ stage1[62];
assign r2 = stage2[63] ^ stage2[62];
assign r3 = stage3[63] ^ stage3[62];

// =====================
// 5. Shift Registers (15-bit)
// =====================
reg [14:0] sr1, sr2, sr3;

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        sr1 <= 0;
        sr2 <= 0;
        sr3 <= 0;
    end
    else
    begin
        sr1 <= {sr1[13:0], r1};
        sr2 <= {sr2[13:0], r2};
        sr3 <= {sr3[13:0], r3};
```

```
    end
end

// =====================
// 6. TMV (Parity Logic)
// =====================
wire g1, g2, g3;

assign g1 = ^sr1;
assign g2 = ^sr2;
assign g3 = ^sr3;

// =====================
// 7. Final XOR Output
// =====================
assign final_response = g1 ^ g2 ^ g3;

endmodule
```